HW1 Design Report

COMS E6998-019: Design of Production Agentic Systems

Part A: Design report

A1. Behavioural contract

Purpose and scope

The agent classifies one Bandit alert at a time in a pinned Python checkout. It determines whether the reported pattern represents a real security issue, returning label 1 or 0 with code evidence when it can classify; otherwise it produces a controlled null-label error or cutoff row. Bandit discovery is separate from model investigation: Bandit discovers the in-scope MEDIUM+ alerts, while the model investigates each discovered alert using only approved read-only tools.

Normal per-alert behaviour

1. The runner discovers and sorts Bandit alerts.
2. For one alert, it creates a disposable agent workspace.
3. The controller receives the alert and initial instructions.
4. The model either requests repository investigation tools or submits JSON.
5. The dispatcher validates every tool request, runs approved requests, and returns the actual observation.
6. The model may use those observations for another decision.
7. A valid submission becomes one classified report row with rationale and evidence.
8. The runner proceeds to the next alert.

Safety and observational integrity

- The agent must never execute code from the target repository.
- The agent must never modify the target repository, including by writing, editing, patching, or deleting files.
- The agent must never fabricate file contents, command results, tool observations, or evidence ranges.
- The agent must never allow the model to read outside its disposable workspace or access `.git`.

- On OWASP BenchmarkPython, the agent must never expose the answer key or any sibling `BenchmarkTest*` case.

The model proposes a tool request. The runtime/dispatcher validates and executes it. Only the dispatcher writes tool observations into the trace.

Controlled outcomes

The system does not silently drop alerts. A malformed submission becomes an error row for that alert, and a denied tool request becomes a refusal observation rather than a crash. A budget cutoff, max-step cutoff, or no-progress stop ends the active run under runtime control. The runner still writes one row for every discovered alert.

A2. State machine

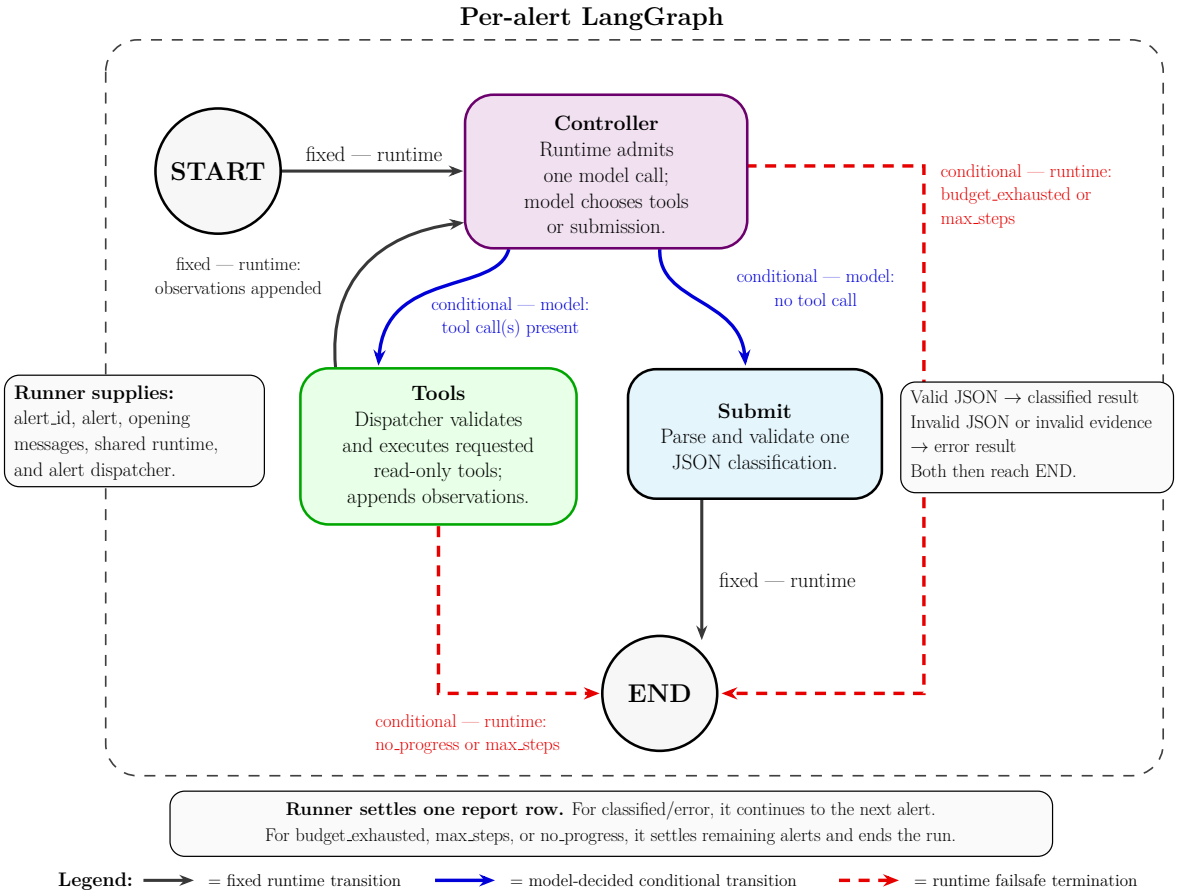


Figure A2. Per-alert reactive state machine. The model decides whether to request investigation tools or submit; the runtime owns tool execution, budget admission, max-step/no-progress failsafes, and terminal consequences.

A3. Nodes and edges

The per-alert graph in Figure A2 has three working nodes. The runner initializes the alert state and invokes the graph; START and END are graph entry and exit markers rather than model-executing nodes. The model selects a requested action, but the runtime controls execution, validation, and all failsafe terminations.

Node	Responsibility
Controller	Checks node-visit and model-call admission limits, invokes the model when permitted, and interprets its response as either tool requests or a proposed final submission.
Tools	Passes requested read-only operations through the runtime dispatcher, which validates arguments, executes allowed requests, and appends actual results as observations. It also checks for repeated identical observation results.
Submit	Parses and validates the proposed JSON classification and evidence; produces a classified result on success or a controlled error result on invalid submission.

Edges and decision ownership.

Transition	Type / owner	Condition or effect
START → Controller	Fixed · Runtime	Begin processing the alert.
Controller → Tools	Conditional · Model	Model requests investigation tools; runtime validates and dispatches them.
Controller → Submit	Conditional · Model	Model returns a final submission instead of a tool request.
Tools → Controller	Fixed · Runtime	Append observations and return for another decision.
Submit → END	Fixed · Runtime	Settle the classification or controlled submission error.
Controller → END	Conditional · Runtime	Stop on `budget_exhausted` or `max_steps`.
Tools → END	Conditional · Runtime	Stop on `no_progress` or `max_steps`.

The runtime checks `max_steps` on each Controller, Tools, and Submit node visit. The token budget is checked immediately before model calls; a permitted call may exceed the limit after its usage is reported. The no-progress detector compares the values of the last three observation results, not just tool names. Section A6 defines the resulting run-level terminal statuses and report-row settlement rules.

A4. Agent state and ownership

`AgentState` exists only for one alert's LangGraph invocation. Because its fields have no custom LangGraph reducer annotations, each node returns the next value for a field; fields that accumulate data are rebuilt manually before being returned.

Field	Update / reducer behavior	Lifetime	Writer	Model visibility
`alert_id`	Initialized once; not changed.	One alert	Runner	Indirect: trace/events and prompt identify the alert; not a separate model-message field.
`alert`	Initialized once; not changed.	One alert	Runner	Yes, through the opening human message that serializes the alert.
`messages`	Node manually rebuilds and returns the full list; controller appends the model reply and tools appends tool-result messages.	One alert	Controller and tools	Yes; this is the conversation sent to the model.
`controller_visits`	Controller returns the prior value plus one.	One alert	Controller	No.
`node_visits`	`_visit()` returns the prior value plus one.	One alert	Controller, tools, and submit through `_visit()`	No.
`next`	Controller overwrites it with `tools` or `submit`.	One alert	Controller	No; it is a runtime routing value.
`status`	Submit writes `classified` or `error`; runtime checks overwrite it for a cutoff.	One alert	Submit and runtime checks	No.

Field	Update / reducer behavior	Lifetime	Writer	Model visibility
`label`	Written once from a valid final submission; `None` for an error.	One alert	Submit	No after submission; it originates from the model's final JSON.
`rationale`	Written once from the final submission or controlled error.	One alert	Submit	No after submission.
`evidence`	Written once from validated final submission; otherwise empty for an error.	One alert	Submit	No after submission.
`observations`	Tools manually rebuilds the accumulated list by appending dispatcher observations.	One alert	Tools	Indirect; each observation is also rendered into a `ToolMessage` in `messages`.

Run-wide state outside the graph. Discovered alerts, report rows, terminal status, and provider usage counters are intentionally not fields of `AgentState`. The runner owns alert discovery, report construction, and run termination, while `Trace` owns the recorded usage ledger. `ModelRuntime` reads that ledger before each model call to enforce the whole-run token budget.

A5. Model-callable tools and dispatcher contract

The model does not directly read the repository. It can only propose one of three registered read-only tool requests: `read_file`, `search`, or `list_directory`. The dispatcher validates each request, executes it against the disposable workspace, records a tool request/result pair in the trace, and returns an observation to the model. Invalid requests become refusal observations rather than crashes.

Tool	Arguments	Normal result and limits	Refusal/error cases
`read_file`	`path`: repository-relative file path. `start`: optional 1-based inclusive start line. `end`: optional 1-based inclusive end line.	Returns UTF-8 text from the requested range. A request returns at most 200 lines; when longer, the result names the next valid `start` line for pagination.	Absolute paths; `..`; paths or symlinks resolving outside the disposable workspace; missing or non-file paths; line numbers below 1; `start` greater than `end`; wrong argument names or types.
`search`	`pattern`: Python regular expression. `glob`: optional repository-relative glob; defaults to Python files.	Returns matching repository-relative paths, line numbers, and line text. Returns at most 10 matches; at the limit the dispatcher asks the model to narrow the pattern. No matches returns *no matches*.	Invalid regular expressions; absolute globs; globs containing `..`; symlinks resolving outside the workspace; malformed or incorrectly typed arguments.
`list_directory`	`path`: optional repository-relative directory path; defaults to the workspace root.	Returns sorted entries with a name and type (file or directory). Returns at most 100 entries. Symlinks escaping the workspace are omitted.	Absolute paths; `..` traversal; symlinks resolving outside the workspace; nonexistent paths; non-directory paths; malformed or incorrectly typed arguments.

Cross-cutting dispatcher behavior

Only registered tools execute. The dispatcher validates the request envelope before indexing it: a request object, a non-empty tool name, and object-valued arguments. Unknown tool names and bad arguments become `ok = false` refusal observations. Requests execute one at a time, and every tool request is immediately followed by its tool result in `trace.jsonl`. Only the dispatcher creates tool observations; the model never invents them. All three tools operate only on the per-alert disposable workspace after withholding has been applied. Therefore, the model cannot access `.git`; on OWASP it also cannot access the answer key or sibling benchmark cases.

A6. Start, Stop, and Termination Policy

A run begins after configuration validation, Bandit discovery, and creation of the shared trace and model runtime. The runner processes alerts sequentially, with each alert invoking the per-alert LangGraph described in A2. In this implementation, one `max_steps` step is one per-alert graph-node visit: Controller, Tools, and Submit each consume one node visit.

Terminal status	Trigger and decision owner	Consequence
`complete`	The runner has processed every discovered Bandit alert without a run-ending failsafe or unhandled exception. Runtime-owned.	`trace.jsonl` ends with terminal status `complete`. Individual alerts may still have controlled `error` rows from malformed model submissions without causing the entire run to terminate with `error`.
`budget_exhausted`	Immediately before each model call, `ModelRuntime` checks whether charged usage is already at or above `budget.max_tokens`. Runtime-owned.	The active alert and every unprocessed alert receive `status = budget_exhausted`, `label = null`, empty evidence, and a matching settlement event. The trace ends with terminal status `budget_exhausted`.
`max_steps`	Before Controller, Tools, or Submit performs work, the runtime checks whether the current alert has reached `budget.max_steps` graph-node visits. Runtime-owned.	The run stops. The active and remaining alerts receive controlled `error` rows with null labels because `report.schema.json` does not define a separate row status for `max_steps`. The trace ends with terminal status `max_steps`.
`no_progress`	After the Tools node appends observations, the runtime compares the three most recent observation results. If all three result values are identical, the run stops. Runtime-owned.	The active and remaining alerts receive controlled `error` rows with null labels. The trace ends with terminal status `no_progress`.
`error`	An unhandled runtime exception occurs, such as a discovery or setup failure not converted into an alert-level controlled result. Runtime-owned.	The runner still writes `report.json`, settles all discovered alerts as `error` rows where possible, and writes exactly one final `error` terminal event.

Budget policy: The `budget.max_tokens` limit applies to the entire run across all alerts, rather than resetting for each alert. Charged usage is calculated from the provider-reported input and output tokens for every `model_call`. Cache-read and cache-write token counts are recorded in the trace when available but are not included in the charged total. The configured `budget.output_allowance` is passed unchanged as the per-call provider output limit, and the budget admission check occurs immediately before each model call. A permitted call can cause reported usage to exceed `max_tokens` because its exact input and output usage is known only after the provider responds. The next model-call admission check then terminates the run as `budget_exhausted`.

Run outcomes versus alert-row outcomes: The report and trace distinguish between individual alert outcomes and whole-run termination. The report-row statuses are `classified`, `budget_exhausted`, and `error`, while the whole-run terminal statuses recorded in the final trace event are `complete`, `budget_exhausted`, `max_steps`, `no_progress`, and `error`. For example, a malformed final model response produces an `error` row for that alert, but the runner normally continues to the next alert and may ultimately finish with terminal status `complete`. Every termination path preserves trace integrity by producing exactly one final terminal event, provider usage totals, one settlement event per report row, and a report containing every discovered Bandit alert.

A7. Workspace policy and withholding boundary

Bandit scans the original `--input` checkout. For each alert, the runner creates a disposable copy for model investigation. The model can inspect only that copy through the dispatcher's read-only tools; the original checkout is never modified. This lets Bandit scan the required scope while keeping the model's view restricted.

Target scope	Withheld files	Reason	Enforcement
Radicale, SGLang, and OWASP BenchmarkPython	`.git/`	A full clone's history may contain commits after the pinned commit, including fixes that would leak information unavailable at that revision.	The workspace constructor physically moves `.git` out of the disposable copy before the model can use any tool.
OWASP BenchmarkPython	Every `expectedresults*.csv` file	These files contain the development-set answer key. Letting the model read them would invalidate the evaluation.	The workspace constructor removes them from the disposable copy before the alert graph starts.
OWASP BenchmarkPython	Every sibling benchmark case except the current case: `testcode/BenchmarkTest*.py` and `templates/web/*/BenchmarkTest*.html`	Benchmark cases have safe twins; another case can reveal the label or solution pattern for the current case.	The runner derives the current case from the Bandit alert. The workspace keeps only files whose stem matches that case and moves all other benchmark-case files out of the model workspace.

Path safety: Withholding is not the only defense. Every model-callable tool resolves requested paths through one shared containment check. It rejects absolute paths and paths containing `..`, resolves symlinks, and rejects any path that would escape the disposable workspace. `read_file`, `search`, and `list_directory` all use this boundary. Therefore, a model cannot bypass withholding with a traversal path, an absolute path, or an escaping symlink.

Workspace lifecycle:

1. Copy the original target checkout to a new temporary directory.
2. Move withheld paths into a private sidecar stash outside the workspace root.
3. Run the per-alert graph with tools rooted at the restricted copy.
4. Move withheld files back into the temporary copy when the alert finishes.
5. Delete the temporary copy unless it was deliberately retained for debugging.

The original `--input` checkout remains unchanged throughout this process. The restriction is enforced by physical file removal and shared path validation, not by asking the model to avoid sensitive files.

Part C: Evidence and README

C2. Results and development-set refinement

Method and cited artifacts

OWASP BenchmarkPython was the development set. I compared its report with the answer key only after each run. The model could not access the answer-key CSV or sibling benchmark cases. Radicale and SGLang were held out, so their results show label counts, token use, and whether the run completed, not measured accuracy. The six new runs are supplied as paired report and trace files with matching configuration files.

Results

The following six runs use GPT-5.4 with Bandit 1.9.4. Full configurations use a 400,000-token run budget and 24 maximum per-alert graph-node visits; low configurations use 5,000 tokens and 12 visits. The table counts report-row labels; "Null" includes both `budget_exhausted` and `error` rows.

Target	Config	Terminal status	Label 1	Label 0	Null
OWASP	Full	`complete`	10	8	0
OWASP	Low	`budget_exhausted`	0	2	16
Radicale	Full	`complete`	3	7	0
Radicale	Low	`budget_exhausted`	0	1	9
SGLang	Full	`max_steps`	1	4	16
SGLang	Low	`budget_exhausted`	0	0	21

Charged tokens / budget: OWASP full 65,032 / 400,000; OWASP low 5,808 / 5,000; Radicale full 131,800 / 400,000; Radicale low 6,345 / 5,000; SGLang full 147,155 / 400,000; SGLang low 5,311 / 5,000. Charged usage is provider-reported input plus output tokens; cached-token counts are traced separately.

OWASP development-set measurement

The new OWASP full run classified all 18 discovered alerts: 10 with label 1 and 8 with label 0, with no null rows. The low-budget run classified two alerts as label 0 and marked the remaining 16 `budget_exhausted`. These figures are prediction counts, not a confusion matrix. Precision, recall, and false-positive/false-negative counts require matching this new full-run report against the withheld development-set answer key; the earlier run's accuracy figures must not be reused without that comparison.

Held-out run interpretation. Radicale full completed all 10 alerts (3 label 1, 7 label 0). SGLang full stopped with terminal status `max_steps`: five alerts were classified (1 label 1, 4 label 0), and 16 alerts have null labels and `error` report-row status. All three low-budget runs ended with `budget_exhausted`. Their charged usage exceeded 5,000 because model-call admission is checked before each call and final usage is recorded after the provider responds. Held-out label counts do not establish accuracy.

Refinements informed by development and trajectory evidence

I improved the prompt and run limits after reviewing development results and real trajectories.

- **Prompt:** I added clearer rules for data-flow, weak-primitive, and XML alerts. I asked the model to use narrow reads, avoid duplicate calls and repeated no-match searches, return evidence-grounded JSON, and submit once it had enough evidence. I also limited it to five tool rounds.
- **Tool output:** `read_file` now returns at most 200 lines and gives the next start line. `search` returns at most ten matches. These limits reduce context growth while allowing the model to request more focused follow-up reads.
- **Budget and steps:** I raised the full token budget to 400,000 after budget-limited runs. For SGLang development runs, I also increased the step limit after an alert reached the earlier limit while still making different investigation requests.
- **Temperature debugging:** I used temperature zero while comparing tool-use paths to reduce sampling variation. This was a debugging experiment, not evidence that temperature alone improved accuracy.

Trace-derived decision loop

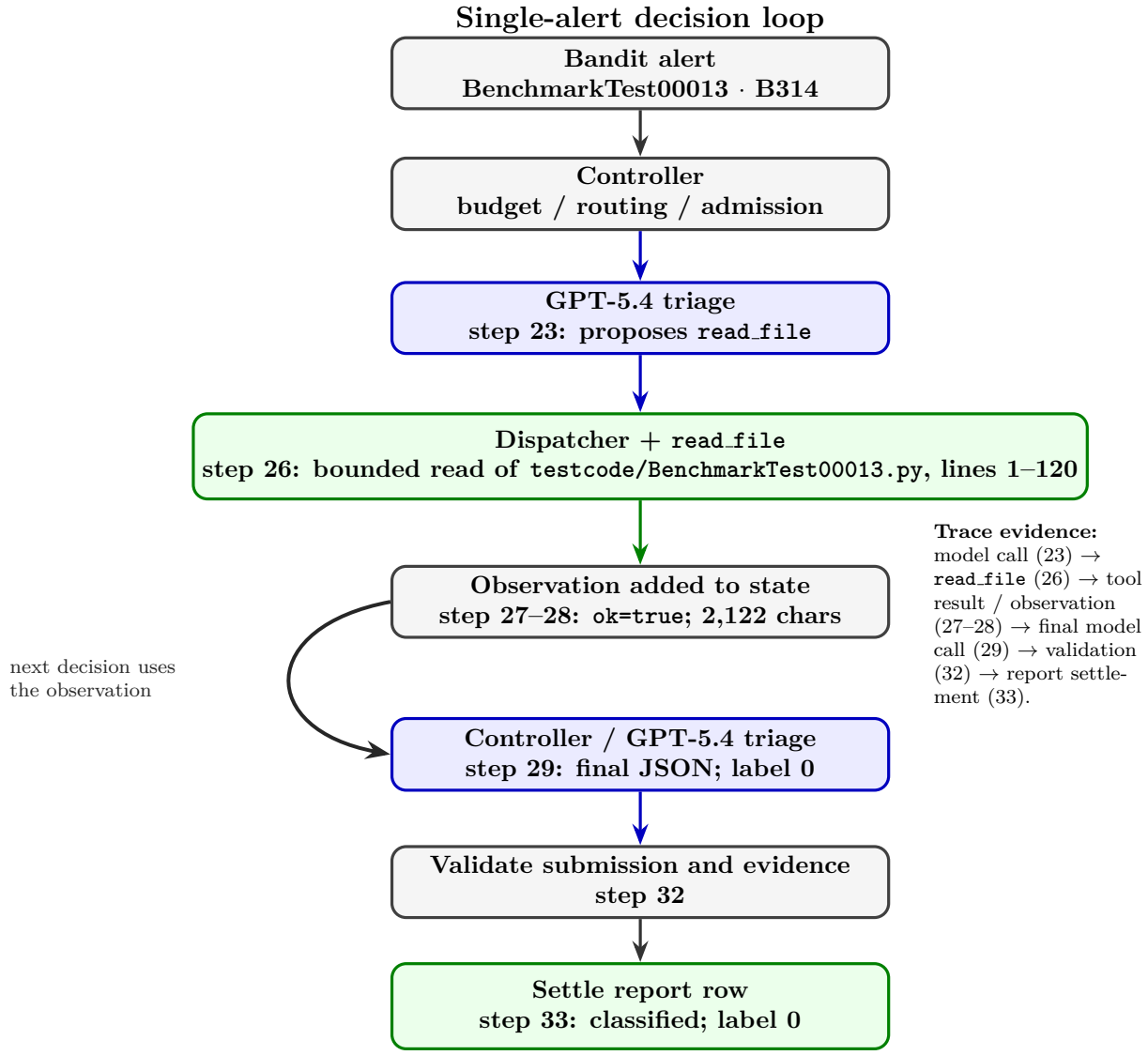


Figure C2. Trace-derived decision loop for one successful OWASP alert. The controller routes the model's bounded file-read proposal through the dispatcher; the returned observation changes the next decision, which submits a validated label-0 report row.
Source: updated OWASP full-run `trace(7).jsonl`, steps 23–33.

C3. Limitations and alternatives

HW1 agent versus plan-and-execute: The HW1 agent lets the model choose the next allowed tool call after each observation. A plan-and-execute agent would instead keep an explicit plan in state, carry out planned steps, and revise the plan when new evidence changes the situation. Planning makes progress and the intended work clearer for a long investigation with dependent steps, but it adds planning and revision work, increasing cost and latency. It also needs rules that distinguish a temporary tool failure from a reason to revise the plan. For the Figure C2 alert, one bounded read gave enough evidence for the next model call to submit a result, so a full plan would add more machinery than necessary.

HW1 agent versus supervisor/worker: The HW1 agent uses one model and the same three bounded investigation tools for every alert. A supervisor/worker design would add a supervisor that routes the alert to a specialist worker and then receives that worker's result. This can help when different alert types need genuinely different expertise, but it adds a routing call and a worker call, increasing cost and latency. It also requires clear rules for what context passes to each worker and who owns the returned result. A bad routing decision can send an alert to the wrong worker without being noticed. Because the HW1 agent uses the same tools for every alert, separate specialist workers would add overhead without a clear benefit.